ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP

HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH – CO5085

THỰC HIỆN BÀI TOÁN PHÂN LOẠI ẢNH TRÊN TẬP DỮ LIỆU CIFAR-10 BẰNG CÁC MÔ HÌNH HỌC SÂU

Giảng viên phụ trách: TS. Lê Thành Sách
TS. Nguyễn Quốc Minh
Lớp – Nhóm: CH01 – DL4DL
Sinh viên thực hiện: Lê Tiến Đạt – 2310653
Nguyễn Lê Thảo Ly – 2312010

Thành phố Hồ Chí Minh, Tháng 10/2026

MỤC LỤC

1 GIỚI THIỆU **5**
1.1 Tổng quan về bài toán phân loại ảnh . . . 5
1.2 Mục tiêu của các bài thực hành . . . 7
1.3 Cấu trúc báo cáo . . . 8

2 TẬP DỮ LIỆU VÀ THIẾT LẬP THỰC NGHIỆM **9**
2.1 Giới thiệu tập dữ liệu . . . 9
2.2 Phân tích và trực quan hóa dữ liệu . . . 10
2.3 Phân chia tập huấn luyện, kiểm định và kiểm thử . . . 12
2.4 Tiền xử lý dữ liệu . . . 12
2.5 Quy trình thực nghiệm chung . . . 13
2.6 Các chỉ số đánh giá . . . 13
2.6.1 Độ chính xác . . . 14
2.6.2 Năng lực và chi phí mô hình . . . 14
2.6.3 Độ ổn định giữa các seed . . . 14
2.6.4 Phân tích lỗi . . . 14

3 BÀI TẬP E1 – PHÂN LOẠI ẢNH VỚI SOFTMAX, MLP VÀ CNN **15**
3.1 Kiến trúc các mô hình . . . 15
3.1.1 Mô hình phân loại Softmax (Softmax classifier) . . . 15
3.1.2 Mạng nơ-ron nhiều lớp (Multi-Layer Perceptron – MLP) . . . 15
3.1.3 Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) . . . 15
3.2 Cài đặt và quy trình huấn luyện . . . 15
3.3 Kết quả thực nghiệm . . . 15
3.4 Nhận xét và thảo luận . . . 15

4 BÀI TẬP E2 – MULTI-HEAD SELF-ATTENTION VÀ TOKENIZATION **16**
4.1 Kiến trúc và cài đặt mô hình . . . 16
4.1.1 Multi-Head Self-Attention tự xây dựng . . . 16

4.1.2 Multi-Head Self-Attention sử dụng PyTorch . . . 17
4.1.3 Mô hình phân loại ảnh sử dụng MSA . . . 17
4.2 Các phương pháp Tokenization . . . 17
4.3 Huấn luyện và đánh giá . . . 18
4.3.1 Quy trình huấn luyện . . . 18
4.3.2 Các thí nghiệm . . . 19
4.3.3 Tiêu chí đánh giá . . . 19
4.3.4 Môi trường thực nghiệm . . . 19
4.4 Kết quả thực nghiệm . . . 20
4.4.1 MSA tự viết và MSA của PyTorch . . . 20
4.4.2 So sánh các phương pháp tokenization . . . 21
4.4.3 Vai trò của positional embedding . . . 23
4.4.4 Trực quan hóa bản đồ chú ý . . . 23
4.4.5 Phân tích lỗi . . . 25
4.5 Nhận xét và thảo luận . . . 25

5 BÀI TẬP E3 – PHÂN LOẠI ẢNH VỚI LSTM VÀ GRU 27
5.1 Phương pháp biểu diễn ảnh dưới dạng chuỗi . . . 27
5.2 Kiến trúc các mô hình . . . 27
5.2.1 Mô hình LSTM . . . 27
5.2.2 Mô hình GRU . . . 27
5.3 Cài đặt và huấn luyện . . . 27
5.4 Kết quả thực nghiệm . . . 27
5.5 Nhận xét và thảo luận . . . 27

6 KẾT LUẬN 28
6.1 Kết quả chính . . . 28
6.2 Hạn chế . . . 28

TÀI LIỆU THAM KHẢO 29

A TÀI NGUYÊN VÀ KHẢ NĂNG TÁI LẬP 30
A.1 Tái lập kết quả E2 . . . 30
A.2 Khai báo sử dụng công cụ AI . . . 31

DANH SÁCH BẢNG

2.1 Thông tin tổng quan về tập dữ liệu CIFAR-10 9
2.2 Phân chia tập dữ liệu trong thực nghiệm 12

4.1 Bốn cách tokenization được triển khai. 18
4.2 Cấu hình huấn luyện của E2. 19
4.3 Độ chính xác kiểm thử (%) và thời gian huấn luyện mỗi epoch (s/ep.) của hai bản MSA. 20
4.4 So sánh bốn tokenizer (MSA tự viết). Độ chính xác validation và kiểm thử (%); thời gian là trung vị giây/epoch huấn luyện; bộ nhớ là đỉnh GPU khi huấn luyện. 21
4.5 Chi phí tính toán một lượt thuận (MSA tự viết). MACs đếm các phép nhân ma trận (Linear, $QK^\top$, AV, tích chập) và khớp với công thức giải tích; ms/bước đo huấn luyện đầy đủ trên RTX 3050 Laptop. 21
4.6 Ablation positional embedding: độ chính xác kiểm thử (%). 23

A.1 Liên kết GitHub Pages và notebook theo từng bài tập 30

DANH SÁCH HÌNH VẼ

2.1 Phân bố số lượng mẫu theo lớp trong tập huấn luyện CIFAR-10 10
2.2 Một số mẫu ảnh trong tập dữ liệu CIFAR-10 11

4.1 Sơ đồ mô hình phân loại ảnh dùng MSA. Chỉ khối MSA (tô xanh) được hiện thực hai cách; các phần còn lại giống hệt nhau. 17
4.2 Độ chính xác validation theo epoch của MSA tự viết (nét liền) và PyTorch (nét đứt), trung bình ba seed. 21
4.3 Loss huấn luyện (trái) và độ chính xác validation (phải) theo epoch của bốn tokenizer; đường là trung bình, vùng mờ là ± std ba seed. 22
4.4 Chú ý của `[CLS]` lên ảnh đầu vào: rollout và khối cuối cho từng tokenizer (seed 0). 24
4.5 Ma trận nhầm lẫn (CNN-stem, seed 0; % theo hàng). 25
4.6 Các ảnh dự đoán sai với độ tin cậy cao nhất. 25

Chương 1

GIỚI THIỆU

1.1 Tổng quan về bài toán phân loại ảnh

Phân loại ảnh (*Image Classification*) là một trong những bài toán cơ bản của thị giác máy tính (*Computer Vision*), với mục tiêu xác định nhãn lớp tương ứng cho một ảnh đầu vào. Bài toán này thuộc nhóm học máy có giám sát (*Supervised Learning*), trong đó mô hình được huấn luyện dựa trên tập dữ liệu gồm các mẫu đầu vào và nhãn tương ứng.

Một tập dữ liệu có giám sát được biểu diễn dưới dạng:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}, \tag{1.1}$$

trong đó N là số lượng mẫu dữ liệu, x_i là đầu vào và y_i là nhãn thực tế (*ground-truth label*) của mẫu thứ i.

Mục tiêu của học máy là học một ánh xạ từ không gian dữ liệu đầu vào đến không gian đầu ra, thay vì xây dựng các quy tắc dự đoán một cách tường minh. Ánh xạ này được biểu diễn tổng quát như sau:

$$f : x \mapsto y. \tag{1.2}$$

Trong đó, hàm f được xấp xỉ thông qua một mô hình có tham số f_θ, với θ là tập hợp các tham số được học từ dữ liệu. Mục tiêu của quá trình huấn luyện là tìm được bộ tham số sao cho đầu ra dự đoán của mô hình phù hợp với nhãn thực tế:

$$\hat{y} = f_\theta(x) \approx y. \tag{1.3}$$

Tùy theo đặc điểm của không gian đầu ra, các bài toán học có giám sát có thể được chia thành hai nhóm phổ biến là phân loại (*Classification*) và hồi quy (*Regression*). Đối với bài toán phân

loại, nhãn đầu ra thuộc một tập hữu hạn gồm K lớp:

$$y \in \{1, 2, \ldots, K\}. \tag{1.4}$$

Các bài toán phân loại thường được chia thành phân loại nhị phân (*Binary Classification*), phân loại đa lớp với một nhãn cho mỗi mẫu (*Single-label Multi-class Classification*) và phân loại đa nhãn (*Multi-label Classification*). Trong phạm vi các bài thực hành, bài toán được xem xét là phân loại đa lớp với một nhãn cho mỗi ảnh.

Đối với dữ liệu ảnh, mỗi mẫu đầu vào x_i được biểu diễn dưới dạng một tensor:

$$x_i \in \mathbb{R}^{C \times H \times W}, \tag{1.5}$$

trong đó C, H và W lần lượt biểu thị số kênh màu, chiều cao và chiều rộng của ảnh.

Khác với bài toán hồi quy, mô hình phân loại đa lớp có thể đưa ra một phân phối xác suất trên tập các lớp. Xác suất có điều kiện để một mẫu đầu vào x thuộc lớp k được biểu diễn bởi:

$$P(y = k \mid x), \quad k = 1, 2, \ldots, K. \tag{1.6}$$

Trong cách tiếp cận sử dụng mạng nơ-ron, một mô hình tham số hóa g_θ tạo ra vector giá trị dự đoán chưa chuẩn hóa (*logits*):

$$\mathbf{z} = g_\theta(x) \in \mathbb{R}^K. \tag{1.7}$$

Đối với bài toán phân loại đa lớp với một nhãn, hàm *Softmax* được sử dụng để chuyển đổi logits thành phân phối xác suất dự đoán:

$$\hat{y}_k = \hat{p}_k = P_\theta(y = k \mid x) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \quad k = 1, 2, \ldots, K. \tag{1.8}$$

Các xác suất dự đoán thỏa mãn $\hat{p}_k \geq 0$ và $\sum_{k=1}^{K} \hat{p}_k = 1$.

Quá trình huấn luyện mô hình là một bài toán tối ưu hóa, trong đó các tham số θ được điều chỉnh nhằm cực tiểu hóa hàm mất mát (*Loss Function*) trên tập dữ liệu huấn luyện. Với bài toán phân loại đa lớp một nhãn, một lựa chọn phổ biến là hàm mất mát *Cross-Entropy*:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_{i,k} \log \hat{y}_{i,k} \tag{1.9}$$

trong đó $\hat{p}_{i,y_i}$ là xác suất mà mô hình dự đoán cho lớp thực tế của mẫu thứ i. Bộ tham số tối ưu được xác định theo:

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta). \tag{1.10}$$

Đối với bài toán phân loại, độ chính xác (*Accuracy*) là một trong những chỉ số đánh giá phổ biến, bên cạnh *Precision*, *Recall* và *F1-score*.

Trên cơ sở đó, các bài thực hành E1–E3 tập trung triển khai, huấn luyện và đánh giá những kiến trúc học sâu khác nhau trên cùng một bài toán phân loại ảnh, nhằm khảo sát ảnh hưởng của cách biểu diễn dữ liệu và đặc điểm kiến trúc đến hiệu quả phân loại và khả năng tổng quát hóa của mô hình.

1.2 Mục tiêu của các bài thực hành

Các bài thực hành E1–E3 được thực hiện nhằm củng cố kiến thức về các kiến trúc học sâu thông qua quá trình xây dựng, huấn luyện và đánh giá mô hình trên bài toán phân loại ảnh có giám sát.

Các mục tiêu cụ thể bao gồm:

- **E1 – Phân loại ảnh bằng Softmax, MLP và CNN:** Xây dựng và huấn luyện ba mô hình phân loại ảnh gồm bộ phân loại Softmax, mạng truyền thẳng nhiều lớp (MLP) và mạng nơ-ron tích chập (CNN). Thông qua việc so sánh độ chính xác, số lượng tham số, quá trình hội tụ và các trường hợp dự đoán sai, bài thực hành nhằm khảo sát sự khác biệt giữa mô hình tuyến tính, mạng truyền thẳng phi tuyến và mạng khai thác cấu trúc không gian của ảnh.

- **E2 – Multi-Head Self-Attention và Tokenization:** Tìm hiểu và hiện thực cơ chế *Multi-Head Self-Attention* (MSA) từ các phép toán cơ bản, đồng thời xây dựng phiên bản sử dụng module có sẵn trong PyTorch để đối chiếu. Tích hợp cơ chế MSA vào mô hình phân loại ảnh, khảo sát ít nhất hai phương pháp chuyển đổi ảnh thành chuỗi token và đánh giá ảnh hưởng của cách biểu diễn token đến độ chính xác và thời gian thực thi.

- **E3 – Phân loại ảnh bằng LSTM và GRU:** Xây dựng mô hình phân loại ảnh dựa trên hai kiến trúc mạng hồi quy *Long Short-Term Memory* (LSTM) và *Gated Recurrent Unit* (GRU). Ảnh đầu vào được chuyển đổi thành chuỗi các thành phần nhằm khảo sát khả năng mô hình hóa quan hệ phụ thuộc trong dữ liệu. Kết quả của hai kiến trúc được đánh giá và so sánh với các mô hình cơ sở từ E1.

Ngoài các mục tiêu riêng của từng bài, quá trình thực hành còn hướng đến việc xây dựng quy trình thực nghiệm nhất quán, bao gồm chuẩn bị dữ liệu, triển khai vòng lặp huấn luyện bằng PyTorch, đánh giá trên các tập dữ liệu độc lập và phân tích kết quả. Các thiết lập huấn luyện và kết quả thực nghiệm được ghi nhận nhằm hỗ trợ việc so sánh và tái lập thí nghiệm.

1.3 Cấu trúc báo cáo

Báo cáo được tổ chức theo trình tự từ phát biểu bài toán, chuẩn bị dữ liệu và thiết lập thực nghiệm đến triển khai mô hình, đánh giá và phân tích kết quả. Nội dung được phân chia thành các phần chính như sau:

- **Giới thiệu:** Trình bày tổng quan về bài toán phân loại ảnh có giám sát, mục tiêu của các bài thực hành và cấu trúc báo cáo.
- **Tập dữ liệu và thiết lập thực nghiệm:** Giới thiệu tập dữ liệu được sử dụng, phương pháp tiền xử lý, cách phân chia tập huấn luyện, xác thực và kiểm tra. Đồng thời, trình bày các thiết lập chung và chỉ số đánh giá phục vụ quá trình thực nghiệm.
- **Bài thực hành E1 – Softmax, MLP và CNN:** Trình bày kiến trúc các mô hình phân loại, phương pháp huấn luyện và kết quả thực nghiệm. Tiến hành so sánh và phân tích sự khác biệt về hiệu quả phân loại và độ phức tạp mô hình.
- **Bài thực hành E2 – Multi-Head Self-Attention và Tokenization:** Trình bày cách hiện thực cơ chế tự chú ý đa đầu, các phương pháp biểu diễn ảnh thành token và kết quả so sánh giữa những phương pháp được triển khai.
- **Bài thực hành E3 – LSTM và GRU:** Trình bày phương pháp biểu diễn ảnh dưới dạng chuỗi, kiến trúc mạng hồi quy và kết quả thực nghiệm so sánh với các mô hình cơ sở.
- **Kết luận:** Tổng hợp các kết quả chính, thảo luận những nhận xét rút ra từ các bài thực hành và chỉ ra những hạn chế của quá trình thực nghiệm.

Chương 2

TẬP DỮ LIỆU VÀ THIẾT LẬP THỰC NGHIỆM

2.1 Giới thiệu tập dữ liệu

Trong bài tập này, nhóm sử dụng tập dữ liệu CIFAR-10 cho bài toán phân loại ảnh đa lớp. CIFAR-10 gồm 60.000 ảnh màu có kích thước 32×32 pixel, được chia thành 10 lớp khác nhau. Mỗi ảnh có ba kênh màu RGB.

Tập dữ liệu gốc gồm 50,000 ảnh dùng cho huấn luyện và 10,000 ảnh dùng cho kiểm thử. Mỗi lớp có số lượng mẫu bằng nhau, với 6,000 ảnh mỗi lớp (tương đương 5.000 ảnh huấn luyện và 1.000 ảnh kiểm thử).

Mười lớp của CIFAR-10 gồm: *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship* và *truck*.

Thuộc tính	**Giá trị**
Tổng số ảnh	60,000
Số ảnh huấn luyện gốc	50,000
Số ảnh kiểm thử	10,000
Kích thước ảnh	32×32
Số kênh màu	3 (RGB)
Số lớp	10

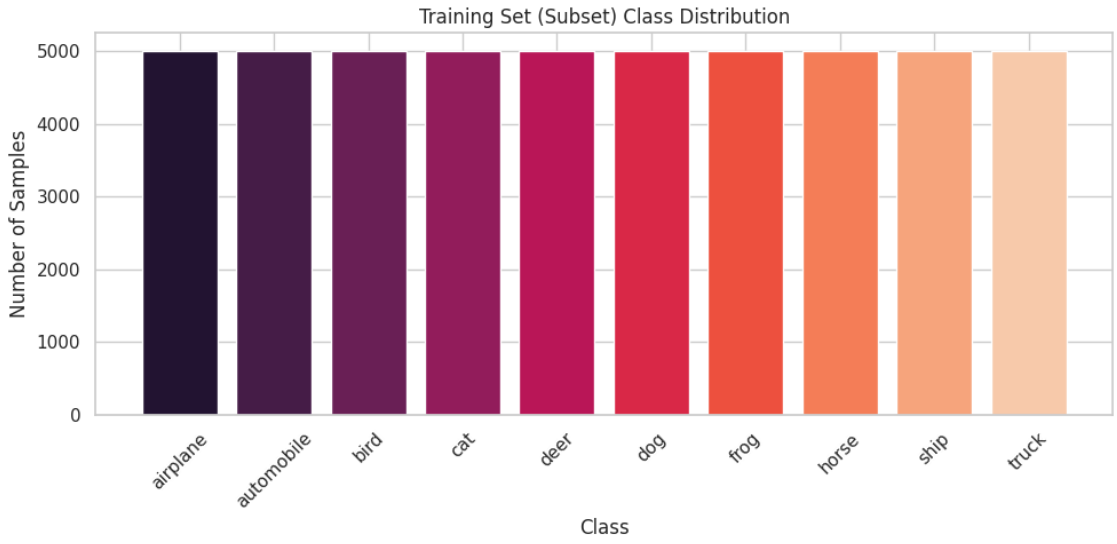
Bảng 2.1: Thông tin tổng quan về tập dữ liệu CIFAR-10

CIFAR-10 được lựa chọn vì quy mô dữ liệu phù hợp với phạm vi bài tập và khả năng tính toán, đồng thời vẫn có độ phức tạp đủ lớn để làm rõ sự khác biệt về hiệu suất giữa ba phương pháp phân loại: bộ phân loại Softmax, mạng MLP và mạng CNN.

2.2 Phân tích và trực quan hóa dữ liệu

Trước khi huấn luyện mô hình, dữ liệu CIFAR-10 được khảo sát thông qua phân bố số lượng mẫu của từng lớp và in ra một số mẫu đại diện cho mỗi lớp.

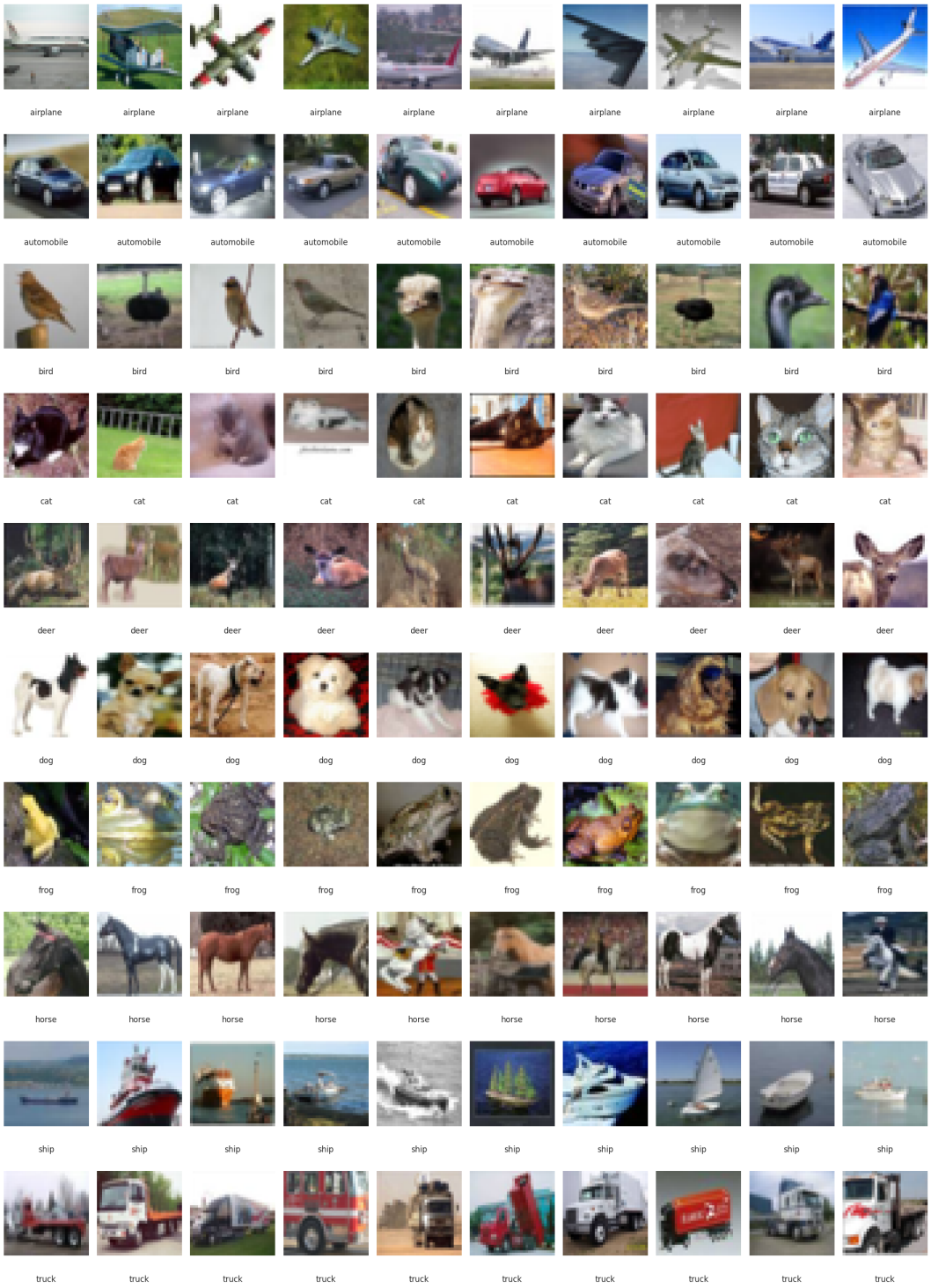
Mục đích của bước này là quan sát đặc điểm trực quan của dữ liệu và kiểm tra mức độ cân bằng giữa các lớp. Hình 2.1 trình bày số lượng mẫu theo từng lớp trong tập huấn luyện gốc.



Hình 2.1: Phân bố số lượng mẫu theo lớp trong tập huấn luyện CIFAR-10

Có thể thấy 10 lớp có số lượng mẫu bằng nhau, với 5.000 ảnh cho mỗi lớp trong tập huấn luyện gốc. CIFAR-10 là tập dữ liệu cân bằng và không cần áp dụng các kỹ thuật xử lý mất cân bằng lớp.

Hình 2.2 minh họa một số ảnh thuộc các lớp khác nhau trong CIFAR-10.



Hình 2.2: Một số mẫu ảnh trong tập dữ liệu CIFAR-10

Các ảnh trong CIFAR-10 có độ phân giải thấp, nhưng vẫn thể hiện sự đa dạng về màu sắc, hình dạng đối tượng và bối cảnh nền. Một số lớp có đặc điểm thị giác tương đối giống nhau, chẳng hạn *cat* và *dog*, hoặc *automobile* và *truck*, làm tăng độ khó của bài toán phân loại.

2.3 Phân chia tập huấn luyện, kiểm định và kiểm thử

Tập huấn luyện gốc của CIFAR-10 gồm 50 000 ảnh được chia *một lần* thành 45 000 ảnh huấn luyện và 5 000 ảnh kiểm định theo kiểu phân tầng: mỗi lớp giữ lại đúng 500 ảnh cho tập kiểm định, nên cả hai tập đều cân bằng lớp. Phép chia dùng seed cố định 42, các chỉ số ảnh được lưu trong tệp `cifar10_split.json` (thư mục `common`) cùng mã băm SHA-256, và mọi bài tập (E1, E2, E3) đọc *cùng một* tệp này; mỗi kết quả huấn luyện ghi lại mã băm để kiểm tra các lần chạy dùng cùng một phép chia. Dữ liệu được tải qua thư viện `datasets` của Hugging Face (`uoft-cs/cifar10`) và chỉ được chấp nhận khi mã băm SHA-256 của toàn bộ ảnh và nhãn trùng với bản TorchVision dùng để tạo phép chia; bản Hugging Face chứa cùng 50 000 ảnh huấn luyện nhưng theo thứ tự khác, nên được sắp xếp lại về thứ tự TorchVision trước khi áp dụng các chỉ số đã lưu. Tập kiểm thử gốc gồm 10 000 ảnh được giữ nguyên, không dùng để chọn mô hình hay siêu tham số, và chỉ được đánh giá sau khi huấn luyện xong.

Bảng 2.2 trình bày cách phân chia dữ liệu trong thực nghiệm.

Tập dữ liệu	Số mẫu	Mục đích
Training	45 000	Huấn luyện mô hình
Validation	5 000	Theo dõi quá trình huấn luyện
Test	10 000	Đánh giá kết quả cuối cùng

Bảng 2.2: Phân chia tập dữ liệu trong thực nghiệm

2.4 Tiền xử lý dữ liệu

Trước khi đưa dữ liệu vào mô hình, các ảnh CIFAR-10 được chuyển đổi và chuẩn hóa nhằm bảo đảm đầu vào có định dạng phù hợp cho quá trình huấn luyện.

Đầu tiên, mỗi ảnh được chuyển sang tensor PyTorch và giá trị pixel được ánh xạ từ khoảng $[0, 255]$ về khoảng $[0, 1]$.

Sau khi chia tập huấn luyện gốc thành 45 000 ảnh huấn luyện và 5 000 ảnh kiểm định, giá trị trung bình và độ lệch chuẩn được tính riêng cho từng kênh màu *chỉ trên 45 000 ảnh huấn luyện*, để thông tin của tập kiểm định và kiểm thử không rò rỉ vào bước chuẩn hóa. Gọi μ_c và σ_c lần lượt là giá trị trung bình và độ lệch chuẩn của kênh màu c. Mỗi giá trị pixel x_c được chuẩn hóa theo công thức:

$$x'_c = \frac{x_c - \mu_c}{\sigma_c}, \tag{2.1}$$

trong đó $c \in \{R, G, B\}$. Các giá trị thu được là

$$\mu \approx (0{,}4918;\ 0{,}4825;\ 0{,}4468), \qquad \sigma \approx (0{,}2471;\ 0{,}2435;\ 0{,}2617).$$

Cùng một phép chuẩn hóa được áp dụng cho tập huấn luyện, tập kiểm định và tập kiểm thử.

Tăng cường dữ liệu (data augmentation). Chỉ trên tập huấn luyện, mỗi ảnh được cắt ngẫu nhiên 32×32 sau khi đệm 4 pixel (random crop) và lật ngang với xác suất 0,5; phép biến đổi được thực hiện sau chuẩn hóa. Tập kiểm định và kiểm thử không bị biến đổi. Các bài tập E1–E3 dùng cùng cấu hình augmentation để so sánh được với nhau; riêng E1 có thêm một lượt chạy không augmentation để đối chiếu.

2.5 Quy trình thực nghiệm chung

Để các bài tập có thể so sánh với nhau, mọi thực nghiệm tuân theo cùng một quy trình:

- **Dữ liệu:** cùng phép chia 45 000/5 000/10 000 và cùng thống kê chuẩn hóa (mục 2.1).
- **Seed:** mỗi cấu hình được huấn luyện với ba seed $0, 1, 2$ (seed điều khiển khởi tạo, thứ tự dữ liệu và augmentation); kết quả là trung bình $\pm$ độ lệch chuẩn mẫu.
- **Số epoch:** 30 epoch cho mọi mô hình của E1–E3.
- **Kết quả chính:** độ chính xác kiểm thử với trọng số của epoch *cuối*; không chọn checkpoint hay siêu tham số theo tập kiểm thử. (E1 ghi thêm kết quả của checkpoint có loss kiểm định thấp nhất.)
- **Tái lập:** mỗi lần chạy lưu một tệp JSON gồm cấu hình, mã băm phép chia, lịch sử huấn luyện, kết quả cuối, thời gian và thông tin môi trường; các bảng và hình trong báo cáo được sinh từ các tệp này bằng mã nguồn trong kho lưu trữ; mỗi bài tập có một notebook Colab nạp lại kết quả đã lưu, sinh lại bảng, hình và kiểm tra chúng khớp với báo cáo, hoặc huấn luyện lại từ đầu (Phụ lục A).

Các siêu tham số riêng của từng bài (bộ tối ưu, tốc độ học, kích thước batch) được nêu trong chương tương ứng.

2.6 Các chỉ số đánh giá

Bốn khía cạnh chính được dùng để so sánh các mô hình: độ chính xác phân loại, năng lực và chi phí mô hình, độ ổn định giữa các seed và các trường hợp dự đoán sai.

2.6.1 Độ chính xác

Độ chính xác là chỉ số chính:

$$\text{Accuracy} = \frac{\text{Số mẫu được dự đoán đúng}}{\text{Tổng số mẫu}} \times 100\%. \tag{2.2}$$

Do CIFAR-10 có phân bố lớp cân bằng, accuracy phù hợp để so sánh hiệu năng tổng thể. Độ chính xác kiểm thử được tính trên 10 000 ảnh; độ chính xác kiểm định chỉ dùng để theo dõi.

2.6.2 Năng lực và chi phí mô hình

Năng lực của mô hình được xem xét qua số tham số huấn luyện được $P = \sum_{i=1}^{L} |\theta_i|$, kết hợp với đặc điểm kiến trúc. Chi phí tính toán được đánh giá bằng thời gian huấn luyện trung vị mỗi epoch, bộ nhớ GPU đỉnh khi huấn luyện và thời gian suy luận (cùng số phép nhân-cộng cho một ảnh ở E2). Các số đo thời gian và bộ nhớ chỉ so sánh được giữa các mô hình chạy trên cùng một máy.

2.6.3 Độ ổn định giữa các seed

Mỗi cấu hình được chạy với ba seed; báo cáo trung bình $\pm$ độ lệch chuẩn mẫu ($n-1$ ở mẫu số). Với $n = 3$, độ lệch chuẩn chỉ là ước lượng thô, vì vậy chỉ các chênh lệch lớn hơn rõ rệt so với độ lệch chuẩn mới được xem là khác biệt thực sự.

2.6.4 Phân tích lỗi

Ma trận nhầm lẫn (chuẩn hóa theo hàng) được dùng để xác định các cặp lớp thường bị nhầm, và một số ảnh bị phân loại sai với độ tin cậy cao được trực quan hóa cùng nhãn thật và nhãn dự đoán để làm rõ kiểu lỗi đặc trưng của từng mô hình.

Chương 3

BÀI TẬP E1 – PHÂN LOẠI ẢNH VỚI SOFTMAX, MLP VÀ CNN

3.1 Kiến trúc các mô hình

3.1.1 Mô hình phân loại Softmax (Softmax classifier)

3.1.2 Mạng nơ-ron nhiều lớp (Multi-Layer Perceptron – MLP)

3.1.3 Mạng nơ-ron tích chập (Convolutional Neural Network – CNN)

3.2 Cài đặt và quy trình huấn luyện

Nội dung đang được bổ sung.

3.3 Kết quả thực nghiệm

Nội dung đang được bổ sung.

3.4 Nhận xét và thảo luận

Nội dung đang được bổ sung.

Chương 4

BÀI TẬP E2 – MULTI-HEAD SELF-ATTENTION VÀ TOKENIZATION

4.1 Kiến trúc và cài đặt mô hình

Bài tập E2 xây dựng bộ phân loại ảnh CIFAR-10 chỉ gồm các khối Transformer encoder, trong đó cơ chế Multi-Head Self-Attention (MSA) được hiện thực theo hai cách khác nhau nhưng nhận *cùng* trọng số khởi tạo: một bản tự viết từ các phép toán cơ bản và một bản bọc `nn.MultiheadAttention` của PyTorch. Nhờ đó, mọi khác biệt về kết quả giữa hai bản chỉ có thể đến từ cách hiện thực.

4.1.1 Multi-Head Self-Attention tự xây dựng

Cho chuỗi N token $X \in \mathbb{R}^{N\times D}$ với $D = 128$ và $h = 4$ head, mỗi head có chiều $d_h = D/h = 32$. Với mỗi head i, các phép chiếu tuyến tính cho $Q_i = XW_i^Q$, $K_i = XW_i^K$, $V_i = XW_i^V$ và

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_h}}\right) V_i, \qquad \text{MSA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O. \tag{4.1}$$

Hệ số $1/\sqrt{d_h}$ giữ phương sai của tích vô hướng ổn định theo d_h, tránh việc softmax bão hòa và gradient bị triệt tiêu [6].

Lớp `MSAScratch` chỉ sử dụng `nn.Linear`, `torch.einsum`, `softmax` và các phép `reshape`/`permute`. Ba phép chiếu Q, K, V được gộp thành một lớp tuyến tính $D \to 3D$ rồi tách head; tích $QK^\top$ và tích với V được viết bằng `einsum` trên chỉ số batch, head và token. Ma trận chú ý (mỗi hàng có tổng bằng 1) có thể được trả về để phục vụ trực quan hóa ở mục 4.4.4. Mỗi khối MSA có $4D^2 + 4D$ tham số (ba phép chiếu, phép chiếu đầu ra và các bias), tức 66 048 tham số với $D = 128$.

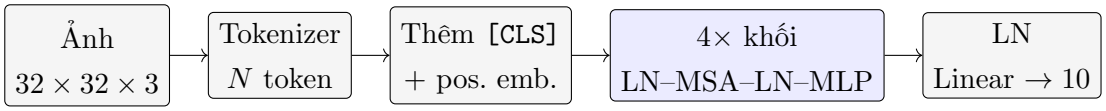
4.1.2 Multi-Head Self-Attention sử dụng PyTorch

Lớp `MSATorch` bọc `nn.MultiheadAttention` (`batch_first=True`) với cùng D, h và cùng bố cục tham số. Bộ trọng số của hai bản được sinh từ cùng một bộ sinh ngẫu nhiên theo seed (Xavier-uniform, bias bằng 0) và được nạp vào hai lớp theo cùng một quy ước; phần còn lại của mô hình (embedding, token `[CLS]`, positional embedding, MLP) cũng được khởi tạo sau khi đặt lại seed, nên hai mô hình bắt đầu huấn luyện từ *đúng cùng một điểm*. Cùng thứ tự dữ liệu và cùng augmentation theo seed được dùng cho cả hai bản.

Tính tương đương được kiểm tra bằng bộ kiểm thử đơn vị (`E2/tests`, 9 kiểm thử, tất cả đạt): với độ chính xác kép, đầu ra, ma trận chú ý và gradient của hai bản trùng nhau trong sai số 10^{-9}; số tham số bằng nhau; hai mô hình đầy đủ (tự viết và PyTorch) có cùng trạng thái ban đầu. Cần lưu ý rằng khi không cần ma trận chú ý, PyTorch có thể dùng các nhân tính toán gộp (fused kernel) nên thời gian chạy và bộ nhớ của hai bản không giống nhau, dù kết quả số học tương đương.

4.1.3 Mô hình phân loại ảnh sử dụng MSA

Mô hình là một Transformer encoder kiểu pre-norm (Hình 4.1): chuỗi token được tạo bởi một tokenizer (mục 4.2), thêm một token `[CLS]` học được ở đầu chuỗi và cộng positional embedding học được; sau đó qua $L = 4$ khối, mỗi khối gồm $x \leftarrow x + \mathrm{MSA}(\mathrm{LN}(x))$ và $x \leftarrow x + \mathrm{MLP}(\mathrm{LN}(x))$, MLP có chiều ẩn $4D = 512$, kích hoạt GELU và dropout 0,1. Đầu phân loại là một lớp tuyến tính áp dụng lên token `[CLS]` sau LayerNorm cuối. Mô hình có khoảng 0,81–0,90 triệu tham số tùy tokenizer (Bảng 4.4), được huấn luyện hoàn toàn từ đầu.



Hình 4.1: Sơ đồ mô hình phân loại ảnh dùng MSA. Chỉ khối MSA (tô xanh) được hiện thực hai cách; các phần còn lại giống hệt nhau.

4.2 Các phương pháp Tokenization

Transformer xử lý chuỗi token 1D nên ảnh 32×32 cần được chuyển thành chuỗi vector D chiều. Bài tập triển khai bốn cách lấy token (Bảng 4.1); mọi mô hình đều thêm token `[CLS]`, nên độ dài chuỗi đưa vào attention là $N + 1$.

Tokenizer	N	$N{+}1$	Cách tạo token
Patch 4×4	64	65	Chia ảnh thành các ô 4×4 không chồng lấn, trải phẳng (48 giá trị) và chiếu tuyến tính lên D (tương đương `Conv2d` có kernel = stride = 4).
Patch 8×8	16	17	Tương tự với ô 8×8 (192 giá trị mỗi token).
Hàng (row)	32	33	Mỗi hàng ảnh ($3 \times 32 = 96$ giá trị) là một token; cấu trúc 2D chỉ còn lại ở thứ tự hàng.
CNN-stem	64	65	Ba lớp tích chập 3×3 (stride 1, 2, 2; BatchNorm và GELU) tạo bản đồ đặc trưng 8×8; mỗi vị trí là một token.

Bảng 4.1: Bốn cách tokenization được triển khai.

Patch 4×4 và 8×8 thuộc kiểu ViT [2], cho phép so sánh ảnh hưởng của số token N (64 so với 16) lên độ chính xác và chi phí tính toán, vì chi phí attention tăng theo N^2. Tokenizer theo hàng là trường hợp token gần như không giữ tính cục bộ 2D. CNN-stem là kiến trúc lai: tương tác cục bộ được học bằng tích chập trước khi vào attention. Các biến thể khác nêu trong đề (patch chồng lấn, token theo kênh, token theo từng pixel với $N = 1024$) không được thực hiện: ba mức $N = 16, 32, 64$ đã đủ để quan sát tác động của độ dài chuỗi, trong khi $N = 1024$ đòi hỏi bộ nhớ và thời gian vượt quá một máy GPU 6 GB trong phạm vi bài tập.

4.3 Huấn luyện và đánh giá

4.3.1 Quy trình huấn luyện

Mọi mô hình E2 dùng cùng dữ liệu và cùng quy trình (dữ liệu, phép chuẩn hóa và augmentation theo mục 2.1–2.5). Cấu hình huấn luyện được tóm tắt ở Bảng 4.2; các siêu tham số được chọn theo công thức phổ biến cho Transformer nhỏ huấn luyện từ đầu và được *cố định cho mọi mô hình* (không tinh chỉnh riêng từng tokenizer trên tập kiểm thử).

Thành phần	Thiết lập
Kiến trúc	Pre-norm, $D = 128$, 4 khối, 4 head, MLP 512, dropout 0,1
Tối ưu	AdamW, lr đỉnh 10^{-3}, weight decay 0,05, cắt gradient 1,0
Lịch lr	Warmup tuyến tính 5% số bước, sau đó cosine về 1% lr đỉnh
Hàm mất mát	Cross-entropy, label smoothing 0,1
Huấn luyện	30 epoch, batch 128, độ chính xác hỗn hợp fp16
Augmentation	Random crop (đệm 4) và lật ngang, chỉ trên tập huấn luyện
Seed	$0, 1, 2$ (ảnh hưởng khởi tạo, thứ tự dữ liệu và augmentation)

Bảng 4.2: Cấu hình huấn luyện của E2.

4.3.2 Các thí nghiệm

Tổng cộng có 30 lần huấn luyện: 4 tokenizer $\times$ 2 bản MSA (tự viết, PyTorch) $\times$ 3 seed $= 24$ lần cho hai so sánh chính (tương đương hiện thực và so sánh tokenizer), cộng 2 tokenizer (Patch 4×4, Hàng) $\times$ 3 seed $= 6$ lần *không có* positional embedding cho thí nghiệm bóc tách (ablation).

4.3.3 Tiêu chí đánh giá

Chỉ số chính là độ chính xác trên 10 000 ảnh kiểm thử chính thức, tính bằng trọng số của epoch *cuối cùng* (không chọn checkpoint theo tập kiểm thử), báo cáo dưới dạng trung bình $\pm$ độ lệch chuẩn mẫu trên 3 seed. Độ chính xác validation chỉ dùng để theo dõi. Ngoài ra báo cáo số tham số, số phép nhân-cộng (MACs) cho một ảnh, thời gian huấn luyện trung vị mỗi epoch và bộ nhớ GPU đỉnh. Hai bản MSA được so sánh theo cặp seed (cùng khởi tạo, cùng thứ tự dữ liệu). Với $n = 3$ seed, các chênh lệch nhỏ hơn độ lệch chuẩn giữa các seed không được xem là có ý nghĩa.

4.3.4 Môi trường thực nghiệm

Mọi lần chạy E2 thực hiện cục bộ, một lần liền mạch (không gián đoạn), khoảng 2,5 giờ, trên máy laptop cắm điện ở chế độ hiệu năng cao, không chạy ứng dụng nặng khác: CPU AMD Ryzen 5 6600H, RAM 16 GB, GPU NVIDIA GeForce RTX 3050 Laptop 6 GB, Python 3.12, PyTorch 2.14.1 (CUDA 12.6). Thời gian đo được chỉ có giá trị so sánh *tương đối* giữa các mô hình trên cùng máy.

4.4 Kết quả thực nghiệm

Mọi số liệu dưới đây là trung bình ± độ lệch chuẩn mẫu trên ba seed $(0, 1, 2)$, độ chính xác tính bằng % trên tập kiểm thử chính thức ở epoch cuối, trừ khi ghi chú khác. Số liệu thô (tệp JSON) nằm trong thư mục `E2/results`; các bảng được sinh tự động bởi `python -m e2lib.analyze`.

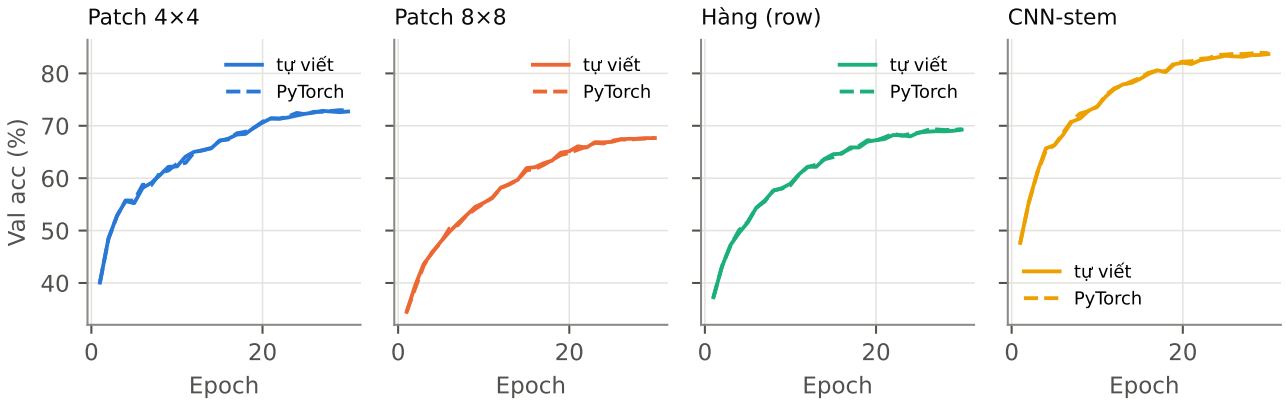
4.4.1 MSA tự viết và MSA của PyTorch

Bảng 4.3 so sánh hai bản hiện thực với cùng tokenizer. Δ là chênh lệch trung bình theo cặp seed (tự viết − PyTorch), tính bằng điểm phần trăm.

Tokenizer	Tham số	Tự viết	PyTorch	Δ	s/ep. (tự viết)	s/ep. (PyTorch)
Patch 4×4	809 354	72,42 ± 0,86	72,41 ± 1,03	+0,02	9,1	8,6
Patch 8×8	821 642	67,09 ± 0,84	67,04 ± 0,83	+0,05	7,0	5,8
Hàng (row)	811 402	68,96 ± 0,41	68,92 ± 0,16	+0,04	6,8	5,7
CNN-stem	896 426	83,00 ± 0,41	83,19 ± 0,54	−0,18	10,3	9,9

Bảng 4.3: Độ chính xác kiểm thử (%) và thời gian huấn luyện mỗi epoch (s/ep.) của hai bản MSA.

Chênh lệch trung bình chỉ từ −0,18 đến +0,05 điểm, nhỏ hơn nhiều so với độ lệch chuẩn giữa các seed (0,16–1,03 điểm) và không có dấu nhất quán. Theo từng seed, chênh lệch lớn nhất là 0,68 điểm (CNN-stem, seed 1); ở Patch 4×4 seed 0 hai bản cho cùng độ chính xác. Các khác biệt còn lại đến từ nhiễu số học (thứ tự phép cộng dấu phẩy động, nhân tính toán khác nhau, fp16) tích lũy qua 30 epoch, không phải từ khác biệt về thuật toán. Đường cong validation của hai bản gần như chồng khít (Hình 4.2). Về chi phí, bản PyTorch nhanh hơn khoảng 4% (Patch 4×4, CNN-stem) đến 17–18% (Patch 8×8, Hàng) mỗi epoch và dùng ít bộ nhớ đỉnh hơn (ví dụ 394 so với 450 MB ở Patch 4×4), nhờ nhân tính toán gộp; khi các phần khác của mô hình chiếm phần lớn thời gian (CNN-stem, nhiều token) lợi thế này nhỏ hơn.



Hình 4.2: Độ chính xác validation theo epoch của MSA tự viết (nét liền) và PyTorch (nét đứt), trung bình ba seed.

4.4.2 So sánh các phương pháp tokenization

Bảng 4.4 so sánh bốn tokenizer với MSA tự viết; Bảng 4.5 bổ sung chi phí tính toán lý thuyết và đo được.

Tokenizer	$N+1$	Tham số	Val	Test	s/epoch	Bộ nhớ (MB)
Patch 4×4	65	809 354	72,73 ± 0,91	72,42 ± 0,86	9,1	450
Patch 8×8	17	821 642	67,65 ± 0,85	67,09 ± 0,84	7,0	265
Hàng (row)	33	811 402	69,25 ± 0,74	68,96 ± 0,41	6,8	323
CNN-stem	65	896 426	83,69 ± 0,32	83,00 ± 0,41	10,3	487

Bảng 4.4: So sánh bốn tokenizer (MSA tự viết). Độ chính xác validation và kiểm thử (%); thời gian là trung vị giây/epoch huấn luyện; bộ nhớ là đỉnh GPU khi huấn luyện.

Tokenizer	$N+1$	MACs / ảnh (triệu)	ms / bước (batch 128)
Patch 4×4	65	55,8	24,8
Patch 8×8	17	14,1	17,4
Hàng (row)	33	27,5	17,5
CNN-stem	65	65,8	28,0

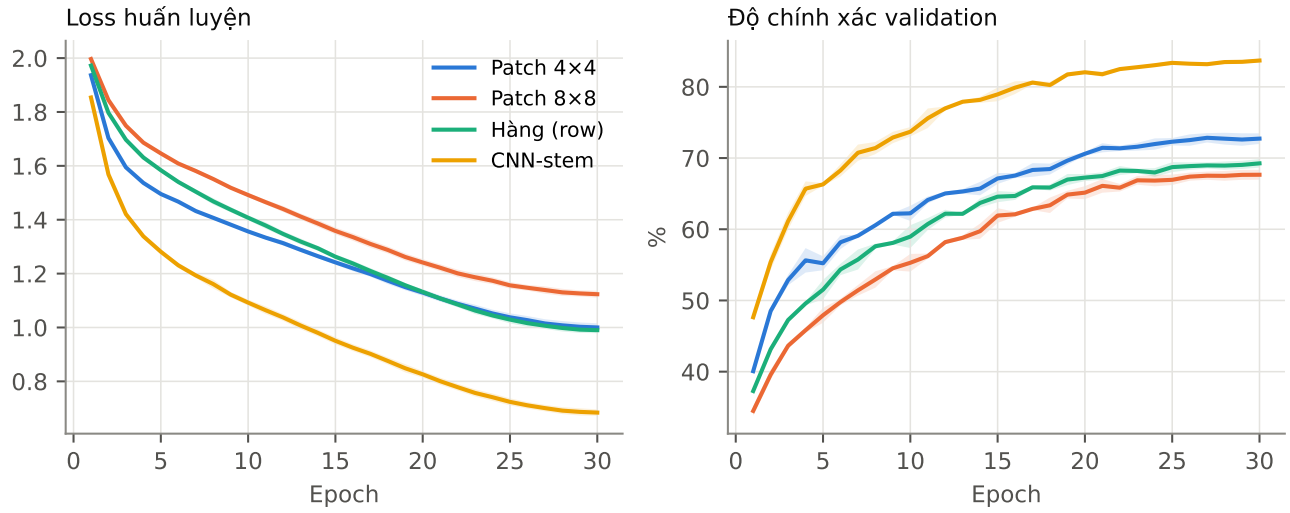
Bảng 4.5: Chi phí tính toán một lượt thuận (MSA tự viết). MACs đếm các phép nhân ma trận (Linear, $QK^\top$, AV, tích chập) và khớp với công thức giải tích; ms/bước đo huấn luyện đầy đủ trên RTX 3050 Laptop.

Về số MACs: công cụ đếm FLOP tự động bỏ sót toàn bộ khối MSA khi chạy bản PyTorch (nhân gộp), nên con số của bản đó thấp hơn (ví dụ 34,5 so với 55,8 triệu ở Patch 4×4). Hai bản thực hiện cùng phép toán, vì vậy bảng dùng số của bản tự viết; số này khớp công thức $L\,(12ND^2 + 2N^2D)$ cộng chi phí embedding (55,84 triệu ở Patch 4×4 tính tay).

Kết quả chính:

- **CNN-stem tốt nhất rõ rệt:** $83{,}00 \pm 0{,}41\%$, hơn Patch 4×4 ($72{,}42 \pm 0{,}86\%$) khoảng 10,6 điểm, trong khi chi phí chỉ tăng khoảng 13% thời gian mỗi epoch. Khoảng cách lớn hơn rất nhiều so với dao động giữa các seed.
- **Patch 4×4 hơn Patch 8×8** (72,42 so với 67,09%; +5,3 điểm) nhưng đắt hơn: số token gấp $\approx 3{,}8$ lần, MACs gấp ≈ 4 lần, thời gian mỗi epoch khoảng 1,3 lần và bộ nhớ gần 1,7 lần.
- **Hàng** (68,96%) nằm giữa hai bản patch; nó cao hơn Patch 8×8 ở cả ba seed (thấp nhất 68,5% so với cao nhất 67,9%), nhưng chênh lệch $\approx 1{,}9$ điểm với $n = 3$ chỉ nên xem là gợi ý.
- Thứ hạng theo kiểm thử: CNN-stem > Patch 4×4 > Hàng > Patch 8×8; thứ hạng theo validation giống hệt.

Hình 4.3 cho thấy CNN-stem hội tụ nhanh hơn và đạt loss huấn luyện thấp hơn hẳn. Ở epoch cuối, độ chính xác huấn luyện (đo trên các batch đã augment) là $\approx$ 71–78% với các mô hình patch/hàng so với 92% của CNN-stem, trong khi khoảng cách huấn luyện–kiểm thử của cả bốn tokenizer đều nằm trong khoảng 4,6–9,1 điểm. Như vậy các mô hình patch/hàng chủ yếu bị hạn chế bởi khả năng khớp dữ liệu trong 30 epoch chứ không phải bởi quá khớp.



Hình 4.3: Loss huấn luyện (trái) và độ chính xác validation (phải) theo epoch của bốn tokenizer; đường là trung bình, vùng mờ là ± std ba seed.

4.4.3 Vai trò của positional embedding

Bảng 4.6 so sánh mô hình có và không có positional embedding (bản không có dùng vector không và cố định; MSA tự viết).

Tokenizer	**Có pos. emb.**	**Không có**	**Δ** (điểm)
Patch 4×4	72,42 ± 0,86	64,78 ± 0,31	−7,65
Hàng (row)	68,96 ± 0,41	62,20 ± 0,65	−6,76

Bảng 4.6: Ablation positional embedding: độ chính xác kiểm thử (%).

Bỏ positional embedding làm giảm độ chính xác 7,65 điểm ở Patch 4×4 và 6,76 điểm ở Hàng, lớn hơn nhiều so với nhiễu giữa các seed ($\leq$ 1,03). Độ chính xác *huấn luyện* cũng giảm ($\approx 77\% \rightarrow 71\%$ ở Patch 4×4), tức mô hình mất khả năng biểu diễn chứ không chỉ khái quát kém hơn. Điều này phù hợp với việc self-attention không phụ thuộc thứ tự token: không có thông tin vị trí, mô hình chỉ thấy một *tập* patch/hàng. Mô hình vẫn đạt 62–65%, vì nội dung từng token (màu, kết cấu, đặc điểm của hàng) đã mang nhiều thông tin về lớp. Thí nghiệm này không chạy cho CNN-stem, nơi các lớp tích chập có thể truyền một phần thông tin vị trí vào token.

4.4.4 Trực quan hóa bản đồ chú ý

Hình 4.4 vẽ, cho bốn ảnh kiểm thử, *attention rollout* [1] (nhân các ma trận chú ý trung bình theo head, đã cộng đường residual, qua các khối) và chú ý của `[CLS]` ở khối cuối, với mô hình tự viết seed 0. Các bản đồ chỉ mang tính gợi ý, không phải bằng chứng nhân quả về căn cứ quyết định của mô hình.



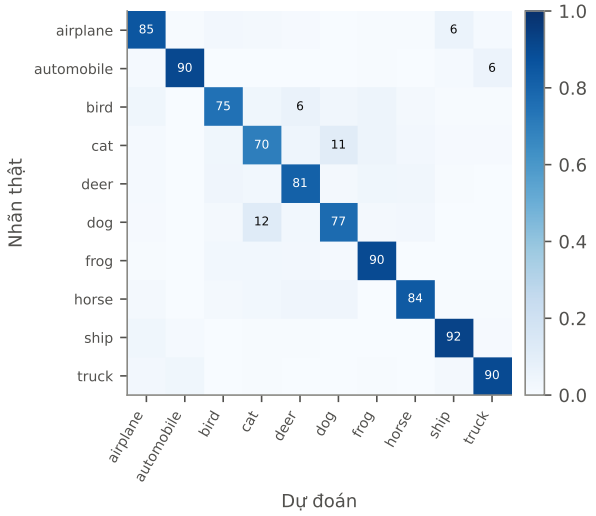
Hình 4.4: Chú ý của `[CLS]` lên ảnh đầu vào: rollout và khối cuối cho từng tokenizer (seed 0).

Quan sát: (i) với Patch 4×4 và CNN-stem, rollout tập trung khá rõ vào thân con ngựa và vùng quanh con tàu và máy bay, còn chú ý khối cuối thường dồn vào một vài điểm cục bộ (đầu, chân, mũi tàu) và đôi khi rơi vào nền; (ii) với Patch 8×8 vùng chú ý thô hơn do chỉ có 16 token; (iii) với tokenizer theo hàng, bản đồ là các dải ngang vì mỗi token là cả một hàng; mô hình ưu

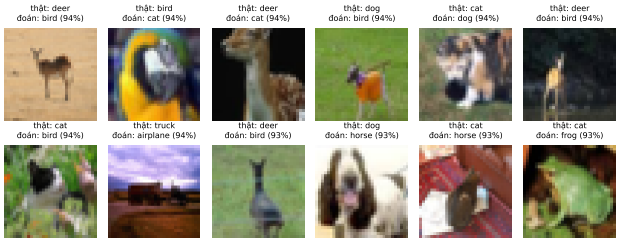
tiên một số hàng nhất định (ví dụ hàng giữa ở ảnh con ngựa) thay vì vật thể theo hình dạng. Với bốn ảnh và một seed, các nhận xét này mang tính minh họa, không được khái quát hóa.

4.4.5 Phân tích lỗi

Với mô hình tốt nhất (CNN-stem, seed 0, kiểm thử 83,34%), Hình 4.5 là ma trận nhầm lẫn chuẩn hóa theo hàng. Lớp dễ nhất là *ship* (92,2%), *automobile* (90,4%), *truck* (89,5%) và *frog* (89,6%); khó nhất là *cat* (69,8%), *bird* (74,8%) và *dog* (77,2%). Các cặp nhầm nhiều nhất là cat↔dog (111 và 120 ảnh), bird→deer (63), airplane→ship (59) và automobile→truck (58), tức các lớp có hình dạng, màu sắc hoặc bối cảnh tương tự; đây cũng là các cặp được dự đoán trước ở mục phân tích dữ liệu. Hình 4.6 cho thấy các ảnh bị dự đoán sai với độ tin cậy cao nhất: nhiều ảnh có vật thể nhỏ, bị che hoặc khó nhận biết ở độ phân giải 32×32.



Hình 4.5: Ma trận nhầm lẫn (CNN-stem, seed 0; % theo hàng).



Hình 4.6: Các ảnh dự đoán sai với độ tin cậy cao nhất.

4.5 Nhận xét và thảo luận

Hiện thực MSA. Bản MSA tự viết (chỉ `Linear`, `einsum`, `softmax`) cho kết quả tương đương bản `nn.MultiheadAttention`: kiểm thử đơn vị xác nhận đầu ra, ma trận chú ý và gradient trùng nhau, và sau 30 epoch chênh lệch độ chính xác nằm trong nhiễu giữa các seed. Lợi ích của bản PyTorch chỉ là tốc độ và bộ nhớ (nhanh hơn 4–18% mỗi epoch trong thiết lập này).

Tokenization quan trọng hơn kích thước attention. Với cùng encoder gần 0,8–0,9 triệu tham số, thay cách tạo token làm độ chính xác thay đổi từ 67% đến 83%. Tokenizer lai CNN-stem vượt xa các tokenizer tuyến tính, phù hợp với việc dữ liệu nhỏ (45 000 ảnh) và huấn luyện từ đầu thiếu thiên kiến quy nạp (inductive bias) về tính cục bộ mà tích chập cung cấp sẵn; điều này nhất

quán với quan sát chung rằng ViT cần nhiều dữ liệu hoặc tiền huấn luyện để vượt CNN [2]. Giữa các tokenizer thuần attention, token nhỏ hơn (Patch 4×4) tốt hơn token lớn (8×8) nhưng chi phí tăng nhanh theo N^2; không có tokenizer nào vừa rẻ vừa chính xác nhất. Tokenizer theo hàng cho kết quả tốt đáng ngạc nhiên so với việc nó bỏ hầu hết cấu trúc 2D, nhưng vẫn kém Patch 4×4.

Thông tin vị trí là cần thiết. Bỏ positional embedding làm giảm ≈ 7 điểm ở cả hai tokenizer được thử, cho thấy thứ tự token mang thông tin không thể thay thế bằng nội dung token.

Hạn chế. (i) Chỉ 3 seed, nên các chênh lệch dưới ≈ 1 điểm (ví dụ Hàng so với Patch 8×8) chưa đủ chắc chắn. (ii) Siêu tham số được giữ cố định cho mọi tokenizer, không tinh chỉnh riêng; một tokenizer có thể hưởng lợi từ lịch huấn luyện khác. (iii) Mô hình nhỏ và chỉ 30 epoch; CNN-stem có độ chính xác huấn luyện cao hơn hẳn và có thể còn cải thiện, trong khi các mô hình khác còn hội tụ chưa hoàn toàn (đường cong vẫn đi lên nhẹ ở epoch cuối). (iv) Thời gian và bộ nhớ chỉ đo trên một GPU xách tay, một lần chạy mỗi cấu hình. (v) CNN-stem có nhiều tham số hơn khoảng 10% (không cân bằng hoàn toàn số tham số). (vi) Bản đồ chú ý chỉ minh họa trên bốn ảnh và một seed. (vii) Các biến thể token trong đề (patch chồng lấn, token theo kênh, từng pixel) chưa được thử.

Chương 5

BÀI TẬP E3 – PHÂN LOẠI ẢNH VỚI LSTM VÀ GRU

5.1 Phương pháp biểu diễn ảnh dưới dạng chuỗi

Nội dung đang được bổ sung.

5.2 Kiến trúc các mô hình

5.2.1 Mô hình LSTM

5.2.2 Mô hình GRU

5.3 Cài đặt và huấn luyện

Nội dung đang được bổ sung.

5.4 Kết quả thực nghiệm

Nội dung đang được bổ sung.

5.5 Nhận xét và thảo luận

Nội dung đang được bổ sung.

Chương 6

KẾT LUẬN

6.1 Kết quả chính

E1. *Sẽ được bổ sung khi có kết quả E1.*

E2. Bản Multi-Head Self-Attention tự viết cho kết quả tương đương `nn.MultiheadAttention` (chênh lệch trung bình $\leq 0{,}18$ điểm, nhỏ hơn nhiễu giữa các seed), trong khi bản PyTorch nhanh hơn 4–18% mỗi epoch. Cách tạo token ảnh hưởng mạnh hơn nhiều: với cùng encoder gần 0,8–0,9 triệu tham số, CNN-stem đạt 83,0%, Patch 4×4 72,4%, Hàng 69,0% và Patch 8×8 67,1% trên CIFAR-10. Bỏ positional embedding làm giảm khoảng 7 điểm.

E3. *Sẽ được bổ sung khi có kết quả E3.*

6.2 Hạn chế

Mỗi cấu hình chỉ được chạy với ba seed và siêu tham số được giữ cố định; các nhận xét cụ thể của từng bài được nêu ở cuối chương tương ứng (mục 4.4.2 cho E2). Phần so sánh tổng hợp giữa E1, E2 và E3 sẽ được hoàn thiện khi có đủ kết quả của ba bài.

TÀI LIỆU THAM KHẢO

[1] Samira Abnar and Willem Zuidema. Quantifying attention flow in transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4190–4197, 2020.

[2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[3] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[4] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[5] Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Phụ lục A

TÀI NGUYÊN VÀ KHẢ NĂNG TÁI LẬP

- GitHub Repository: `https://github.com/broistg/deep-learning-CO5085`
- Trang tổng quan (GitHub Pages): `https://broistg.github.io/deep-learning-CO5085/`
- Trọng số và logit kiểm thử: GitHub Release `weights-v1` của kho lưu trữ (mã băm SHA-256 trong `common/release_manifest.json`).

Bài tập	GitHub Pages	Notebook Google Colab
E1 – Softmax, MLP và CNN	Trang E1	Mở bằng Colab
E2 – Multi-Head Self-Attention và Tokenization	Trang E2	Mở bằng Colab
E3 – LSTM và GRU	Trang E3	*Sẽ bổ sung*

Bảng A.1: Liên kết GitHub Pages và notebook theo từng bài tập

A.1 Tái lập kết quả E2

Notebook tái lập (khuyến nghị)

Notebook `E2/notebooks/E2.ipynb` (mở bằng Google Colab) gọi mã trong kho lưu trữ và không chứa logic riêng. Biến `MODE` ở ô đầu tiên chọn một trong hai chế độ:

- `"load"` (mặc định): nạp 30 tệp JSON kết quả đã lưu, sinh lại các bảng và hình, rồi *kiểm tra* các bảng sinh lại trùng từng ký tự với các bảng trong báo cáo. Chạy được trên Colab miễn phí

chỉ với CPU, mất vài phút. Ma trận nhầm lẫn và các ảnh lỗi cần thêm logit kiểm thử, được tải từ GitHub Release `weights-v1` và kiểm tra bằng mã băm SHA-256.

- `"rerun"`: huấn luyện lại toàn bộ 30 lần chạy từ đầu (cần GPU; khoảng 2,5 giờ trên RTX 3050 Laptop), giữ kết quả đã công bố trong `E2/results_published/` để đối chiếu. Do GPU, cuDNN và độ chính xác hỗn hợp khác nhau, kết quả chạy lại không trùng từng bit; chúng được đánh giá bằng trung bình mỗi cấu hình so với độ lệch chuẩn đã công bố.

Dữ liệu CIFAR-10 được tải qua thư viện `datasets` (Hugging Face) và chỉ được chấp nhận khi mã băm SHA-256 của toàn bộ ảnh và nhãn trùng với bản TorchVision dùng để tạo phép chia cố định (bản Hugging Face lưu tập huấn luyện theo thứ tự khác nên được sắp xếp lại; TorchVision là nguồn dự phòng).

Chạy bằng dòng lệnh

Mã nguồn E2 nằm trong thư mục `E2/` của kho lưu trữ; các lệnh dưới đây chạy từ thư mục gốc của kho (`PYTHONPATH=.:E2` hoặc `pip install -e .`). Mỗi bước độc lập với bước sau; kết quả JSON đã chạy xong được giữ trong `E2/results/`, nên có thể bỏ qua bước huấn luyện và chỉ sinh lại bảng, hình.

1. Kiểm thử đơn vị (tương đương MSA, hình dạng token, phép chia dữ liệu): `python -m unittest discover -s E2/tests -v`
2. Đo nhanh thời gian, bộ nhớ, MACs: `python -m e2lib.bench`
3. Chạy toàn bộ 30 lần huấn luyện (có thể dừng và chạy tiếp, bỏ qua các lần đã xong; khoảng 2,5 giờ trên RTX 3050 Laptop): `python -m e2lib.plan`
4. Bảng và hình học tập: `python -m e2lib.analyze`
5. Bản đồ chú ý: `python -m e2lib.viz -seed 0`

Môi trường của các kết quả trong báo cáo: Python 3.12, PyTorch 2.14.1 (CUDA 12.6), `timm` 1.0.30, GPU NVIDIA GeForce RTX 3050 Laptop 6 GB, CPU AMD Ryzen 5 6600H, RAM 16 GB; thông tin này cũng được ghi trong từng tệp JSON.

A.2 Khai báo sử dụng công cụ AI

Nhóm lên ý tưởng và tự viết bản gốc của mã. Công cụ AI (ChatGPT / Claude) hỗ trợ cải thiện và rà soát mã, xây dựng bộ kiểm thử và bản nháp báo cáo. Nhóm đã tự chạy toàn bộ thực nghiệm, kiểm tra các số liệu đưa vào báo cáo với tệp kết quả gốc, và chịu trách nhiệm về toàn bộ nội dung.